

Task 13

GitHub Actions CI/CD Pipeline for ECS Deployment using Terraform

Umar Satti

Table of Contents

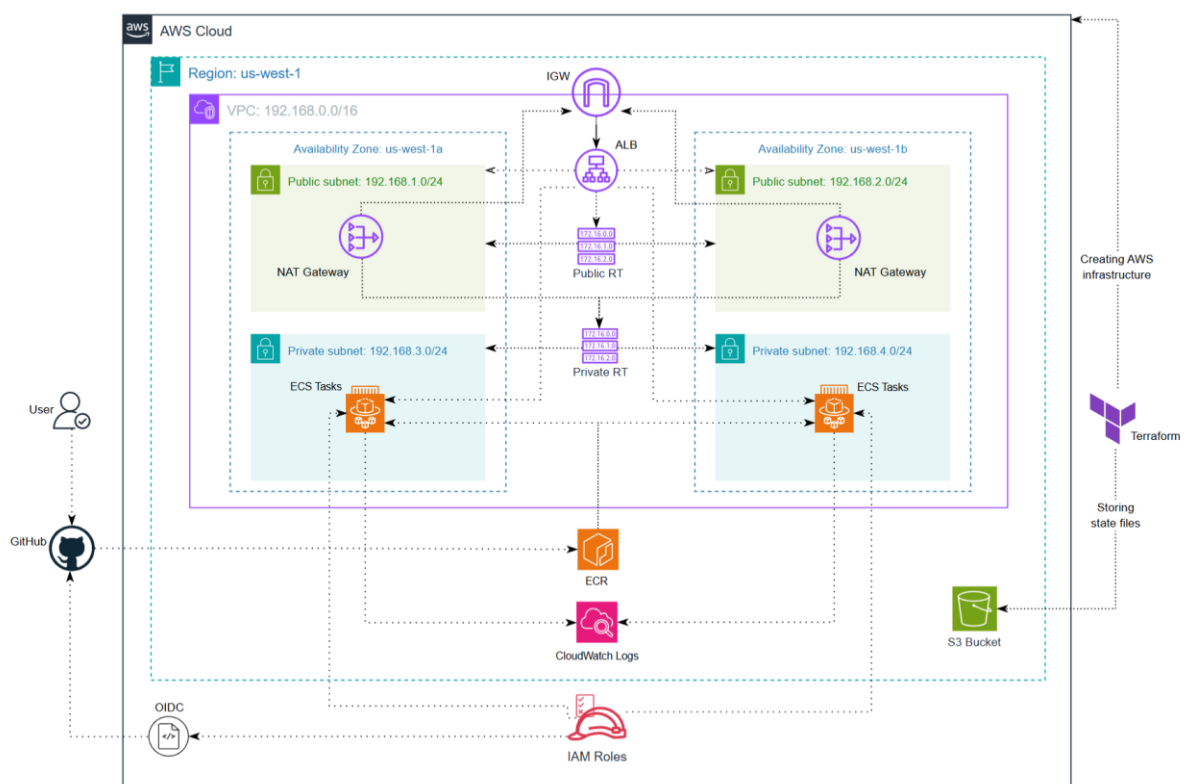
1. Task Description	3
2. Architecture Diagram	3
3. Project Structure	4
3.1 Application Files	4
3.2 Terraform Infrastructure Code	5
3.2.1 Root Terraform Files	5
3.2.2 Terraform Modules Directory	6
3.2.3 Supporting Terraform Files	9
3.3 Terraform Commands	10
4. Validate Infrastructure in AWS	11
4.1 VPC and Networking Validation	11
4.2 IAM Validation	13
4.3 ECR Validation	14
4.4 ECS Validation	14
4.6 Application Load Balancer Validation	15
5. GitHub Actions CI/CD Pipeline	17
5.1 GitHub Actions Workflow Overview	17
5.2 OIDC-Based Authentication with AWS	19
5.3 Workflow Execution Steps	19
5.4 Manual Workflow Trigger	22
5.5 Validation	23
6. Clean Up	26
7. Troubleshooting	26

1. Task Description

This project implements a secure CI/CD pipeline using GitHub Actions, Terraform, and AWS ECS Fargate. All infrastructure including VPC networking, IAM roles, GitHub Workflows, Amazon ECR, ECS cluster, and Services were provisioned using Terraform.

The pipeline uses GitHub-hosted runners and OIDC-based IAM role access to build Docker images, push them to ECR, and deploy updates to Amazon ECS Fargate.

2. Architecture Diagram



3. Project Structure

This project is organized into application source files and Terraform infrastructure code. The structure separates application logic, CI/CD configuration, and infrastructure provisioning for clarity and maintainability.

3.1 Application Files

These files define the Node.js application that is containerized and deployed to ECS.

3.1.1 `app.js`

- Main application entry point.
- Uses Express to serve static files from the public directory.
- Listens on port 3000 (or environment variable PORT).
- Includes a `/health` endpoint for basic health checks.

3.1.2 `package.json`

- Defines application metadata and dependencies.
- Specifies Node.js runtime version.
- Includes scripts for starting the application.
- Lists express as the primary dependency.

3.1.3 `public/index.html`

- Static HTML file served by the Node.js application.
- Displays a simple UI confirming successful deployment.
- Used to verify application availability after ECS deployment.

3.1.4 `Dockerfile`

- Defines a multi-stage Docker build.
- Installs production dependencies using Node.js Alpine image.
- Copies application source and static files.
- Exposes port 3000 and starts the Node.js application.

3.1.5 GitHub Actions Workflow (`.github/workflows/deploy.yml`)

- Declarative GitHub Actions pipeline definition.
- Automates build, test, Docker image creation, ECR push, and ECS deployment.
- Configured to run on a GitHub-hosted ubuntu runner.
- Uses OIDC for AWS access.

3.2 Terraform Infrastructure Code

This directory contains Terraform configuration used to provision and manage all AWS infrastructure required for VPC, IAM, ECR, ECS, and Load balancing. The setup follows a modular design to improve reusability and clarity.

3.2.1 Root Terraform Files

These files act as the entry point for Terraform and orchestrate all infrastructure modules.

1. terraform.tf

- Defines Terraform and AWS provider requirements.
- Locks the AWS provider version to 6.27.0.
- Configures **remote state storage** using an S3 backend.
- Terraform state is stored in:
 - **S3 Bucket:** umarsatti-terraform-state-file-s3-bucket-sandbox
 - **State File Path:** Task-13-GitHub-Actions/terraform.tfstate
 - **Region:** us-west-1
- Enables state encryption and locking to prevent concurrent modifications.
- Configures the AWS provider region as us-west-1.

2. main.tf

- Serves as the central orchestration file.
- Calls all Terraform modules required for the solution:
 - **VPC** – Networking and security components.
 - **IAM** – Roles and policies for ECS, and GitHub OIDC.
 - **ECR** – Docker image repository.
 - **ALB** – Application Load Balancer for ECS services.
 - **ECS** – ECS cluster, task definition, and service.
- Passes shared outputs (such as subnets, security groups, and IAM roles) between modules.
- Ensures proper dependency flow across infrastructure components.

3. variables.tf

- Declares all input variables required by the root module.
- Groups variables logically by modules i.e. VPC, IAM, ECR, ALB, and ECS.
- Enables flexible configuration without modifying core Terraform code.

4. terraform.tfvars

- Supplies values for all declared variables.

- Defines environment-specific settings such as:
 - VPC CIDR and naming
 - EC2 AMI and instance sizes
 - ECS task sizing and launch type
 - ALB listener and target group configuration
- Allows easy reuse of the same Terraform code across environments.

5. outputs.tf

- Exposes key infrastructure outputs after Terraform execution.
- Provides ALB DNS name for accessing the deployed application.
- Useful for validation, troubleshooting, and CI/CD integration.

3.2.2 Terraform Modules Directory

1. VPC Module

The VPC module provisions the complete networking layer, including subnets, routing, NAT gateways, and security groups.

vpc/main.tf

- Defines a custom VPC with DNS support enabled.
- Creates:
 - **Two public subnets** across us-west-1a and us-west-1b
 - **Two private subnets** across the same availability zones
- Provisions:
 - Internet Gateway for public subnets
 - Elastic IPs and NAT Gateways (one per public subnet)
- Configures:
 - Public route tables with internet access
 - Private route tables with outbound internet access via NAT
- Implements security groups for:
 - Application Load Balancer (HTTP on port 80)
 - ECS Tasks (port 3000 from ALB SG)
- Uses local variables to simplify subnet and routing mappings.

vpc/variables.tf

- Declares input variables required to configure the VPC:
 - CIDR block

- Naming
- Internet and routing parameters

vpc/outputs.tf

- Exposes key networking resources to other modules:
 - VPC ID
 - Public and private subnet IDs
 - Security group IDs for ALB, EC2, and ECS

2. IAM Module

The IAM module provisions roles and policies required by GitHub Action workflows and ECS tasks.

iam/main.tf

- Creates an **ECS Task Execution role** with:
 - Trust relationship for ECS tasks
 - Custom ECS permissions loaded from an external policy document.
- Creates a **GitHub IAM role** with:
 - Trust relationship using GitHub OIDC in AWS.
 - Custom ECS and ECR permissions from an external policy document.

iam/variables.tf

Declares input variables for ECS task execution role and GitHub Actions IAM role.

- ECS task role and policy names
- GitHub role and policy names
- GitHub repository and branch

iam/outputs.tf

Exposes ECS and GitHub role ARNs.

3. ECR Module

The ECR module provisions a private container registry for application images.

ecr/main.tf

Creates a private ECR repository. Configures Image tag mutability, AES-256 encryption at rest, and forced deletion to simplify cleanup. Disables automatic image scanning on push.

ecr/variables.tf

Defines repository name, mutability, and encryption type.

ecr/outputs.tf

Outputs the ECR repository URI used by ECS task definitions.

4. ALB Module

The ALB module provisions an Application Load Balancer to expose the ECS service.

alb/main.tf

Creates an internet-facing Application Load Balancer.

- Attaches:
 - ALB security group
 - Public subnets
- Configures:
 - Target group using **IP-based targets** (required for Fargate)
 - HTTP listener on port 80 forwarding traffic to the target group

alb/variables.tf

- Defines inputs for load balancer, target group, and listener configuration.
- Accepts VPC, subnet, and security group references.

alb/outputs.tf

Outputs:

- Load balancer DNS name
- Load balancer ARN
- Target group ARN

5. ECS Module

The ECS module provisions the container orchestration layer using AWS Fargate.

ecs/main.tf

- Creates a CloudWatch log group for container logs.
- Defines an ECS task definition:
 - Uses Fargate launch type
 - Pulls the latest image from ECR

- Exposes port 3000
 - Sends logs to CloudWatch
- Provisions an ECS cluster with:
 - Container Insights enabled
 - Execute Command support
- Creates an ECS service:
 - Runs two tasks using rolling deployments
 - Deploys tasks in private subnets
 - Attaches ECS tasks to the ALB target group
 - Uses security groups for controlled inbound traffic

ecs/variables.tf

- Defines inputs for:
 - Logging, task definition, and cluster configuration
 - ECS networking, security group, and target group references

ecs/outputs.tf

- No outputs are defined for this module.

3.2.3 Supporting Terraform Files

This section describes JSON policy documents used by Terraform to define IAM permissions for ECS and GitHub Actions.

GitHub IAM Policy (github_policy.json)

This policy is attached to the IAM role used by GitHub Actions. This allow GitHub to:

- Describe and update ECS services
- Describe ECS tasks and task definitions
- Grant permissions to pass IAM roles to ECS tasks (iam:PassRole).
- Provide ECR actions

These permissions enable GitHub Actions pipeline to trigger ECS deployments.

ECS Task Execution Policy (ecs_policy.json)

This policy is attached to the ECS Task Execution role. Allows ECS tasks to:

- Authenticate with Amazon ECR
- Pull container images from ECR
- Write application logs to CloudWatch Logs

It ensures ECS tasks can start, run, and log correctly while following the principle of least privilege.

3.3 Terraform Commands

1. Terraform init

Initializes the Terraform working directory by downloading required providers and modules, configuring the remote backend, and preparing Terraform to manage the infrastructure.

2. Terraform validate

Checks the Terraform configuration for syntax and logical errors without creating resources, ensuring the configuration is valid and consistent.

3. Terraform plan

Creates an execution plan that previews the AWS resources Terraform will create and any changes that will be applied, allowing review before deployment.

4. Terraform apply

The terraform apply command provisions the AWS infrastructure as defined in the Terraform configuration. Terraform creates all required resources including VPC components, ECS, ALB, IAM roles, and ECR.

Terraform also outputs key infrastructure details for later use:

- **ALB DNS:** alb-ecs-github-actions-380484820.us-west-1.elb.amazonaws.com

These outputs are used for accessing the deployed application through the ALB.

```
Apply complete! Resources: 37 added, 0 changed, 0 destroyed.
```

```
Outputs:
```

```
alb_dns = "alb-ecs-github-actions-380484820.us-west-1.elb.amazonaws.com"
```

```
PS D:\Cloudelligent\Task-13-GitHub-Actions\terraform> █
```

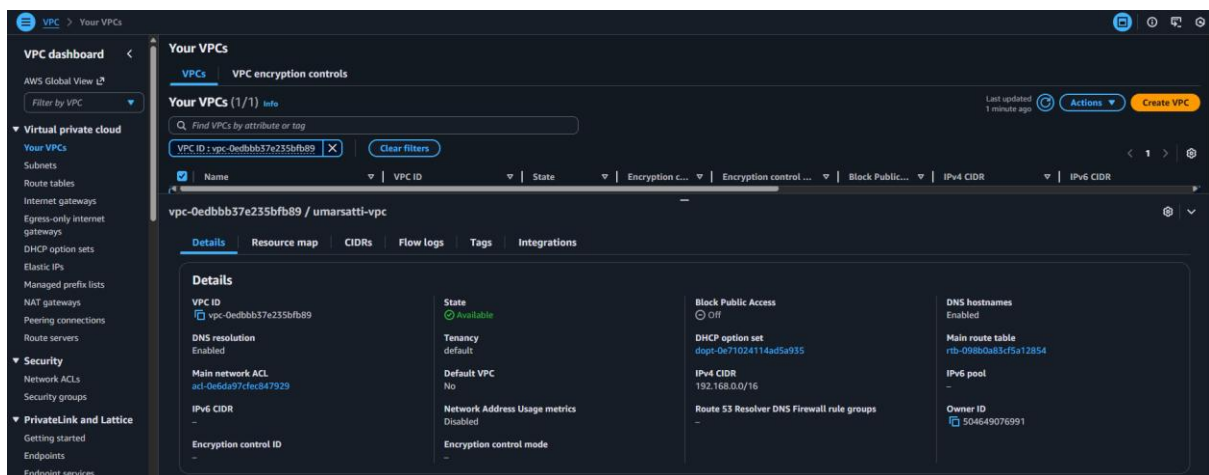
4. Validate Infrastructure in AWS

This section validates that all AWS resources created using Terraform are provisioned correctly and functioning as expected.

4.1 VPC and Networking Validation

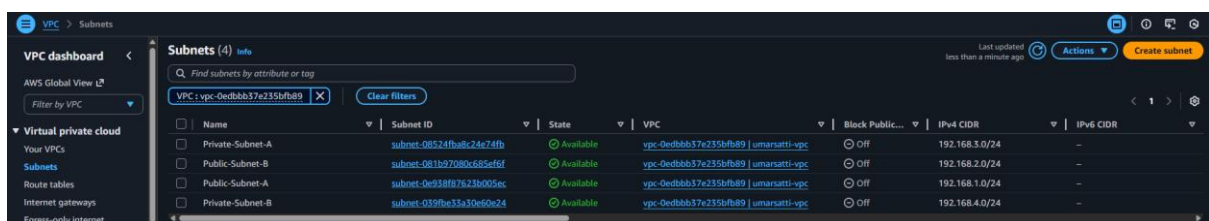
1. Verify VPC

- Open the AWS Management Console.
- Navigate to **VPC** using the search bar.
- Select **Your VPCs** and verify:
 - VPC created by Terraform exists.
 - Correct IPv4 CIDR block as defined in Terraform.
 - DNS hostnames and DNS resolution are enabled.
 - VPC name matches the Terraform configuration.



2. Verify Subnets

- Navigate to **Subnets** in the VPC console.
- Confirm creation of all subnets.



Public Subnets (Used by Load balancer)

- Public Subnet A (us-west-1a) - 192.168.1.0/24
- Public Subnet B (us-west-1b) - 192.168.2.0/24

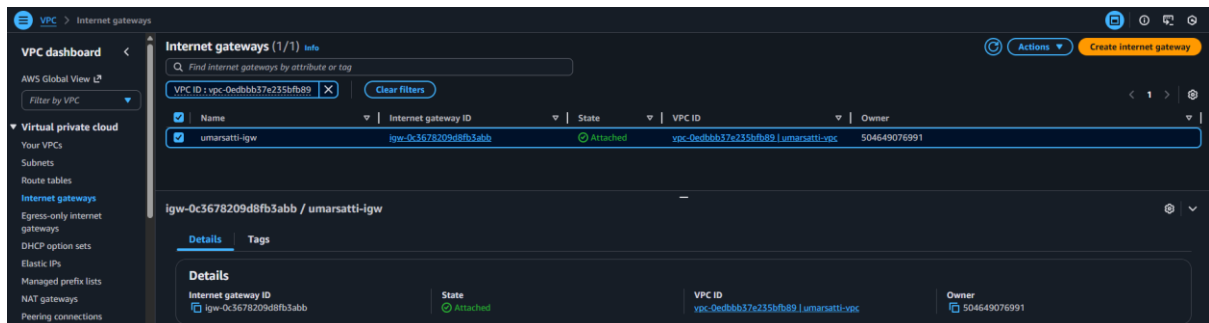
Private Subnets (Used by ECS tasks)

- Private Subnet A (us-west-1a) - 192.168.3.0/24
- Private Subnet B (us-west-1b) - 192.168.4.0/24
- Private subnets are used by ECS Fargate tasks.

3. Internet Gateway

Navigate to **Internet Gateways** in the VPC console. Confirm the following:

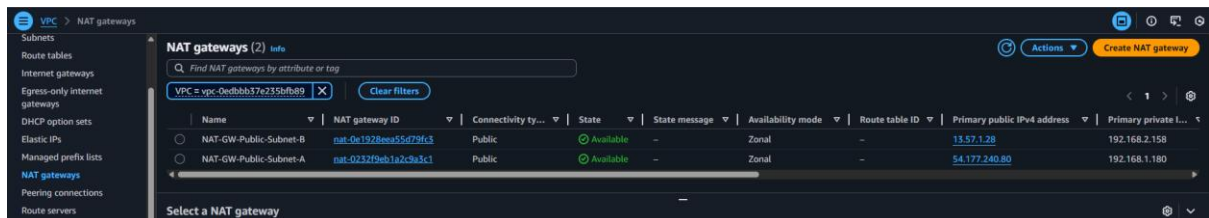
- Internet gateway exists (example: umarsatti-vpc).
- **Attached** status to the VPC.



4. NAT Gateway

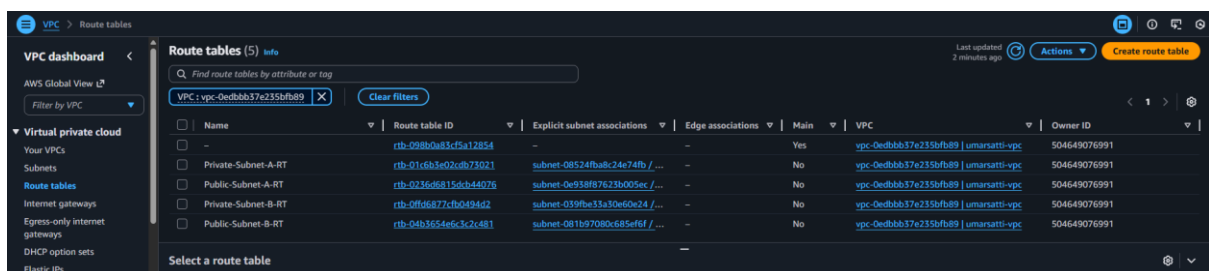
Navigate to **NAT Gateways** in the VPC console. Confirm the following:

- NAT Gateway exists in each of the public subnets (A and B).
- Status: **Available**.
- **Elastic IPs** are attached to **NAT Gateways**.



5. Route Tables

Navigate to **Route Tables** in the VPC console. Confirm that two public and two private route tables are created.



Public route table should be associated with public subnets and contain the route 0.0.0.0/0 to Internet Gateway.

Private route table should be associated with public subnets and contain the route 0.0.0.0/0 to NAT Gateway.

6. Security Groups

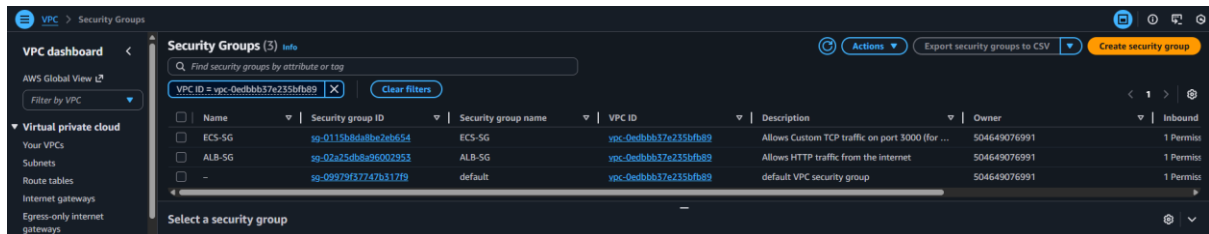
- Navigate to **Security Groups** in the VPC console.

ALB Security Group

- **Inbound rules:** HTTP (80) TCP traffic from 0.0.0.0/0
- **Outbound rules:** All traffic allowed (0.0.0.0/0)

ECS Service Security Group

- **Inbound rules:** TCP 3000 from ALB security group
- **Outbound rules:** All traffic allowed (0.0.0.0/0)

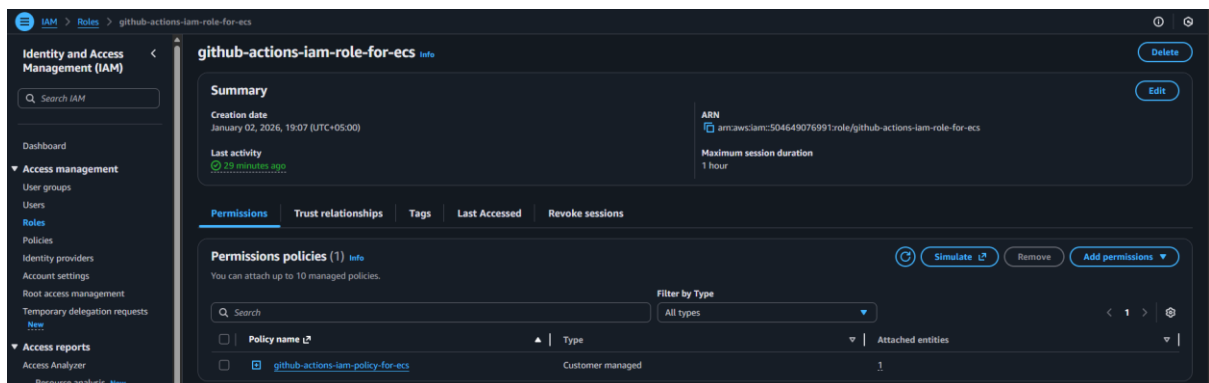


4.2 IAM Validation

1. GitHub Actions IAM Role

Navigate to **IAM** console and select **Roles**.

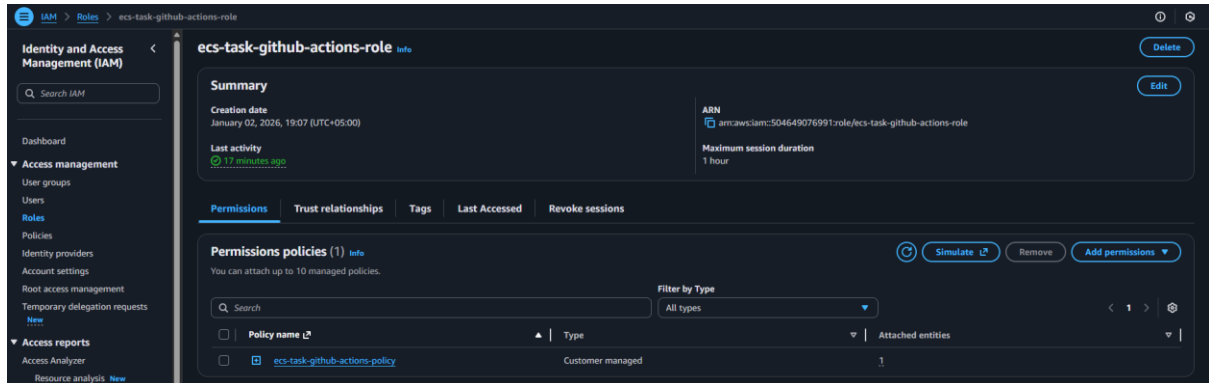
- Select **github-actions-iam-role-for-ecs** IAM role.
- Confirm the following:
 - Has inline policy **github-actions-iam-policy-for-ecs**.
 - Attached policy matches **github_policy.json**.



2. ECS Task Execution Role

Navigate to **IAM** console and select **Roles**.

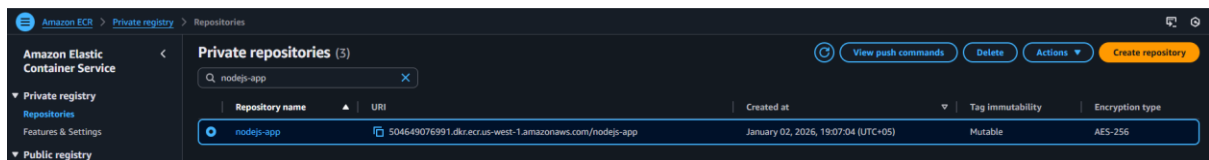
- Select **ecs-task-github-actions-role** IAM role.
- Verify ECS task execution role. Confirm the following:
 - Attached policy matches **ecs_policy.json**.
 - Permissions allow ECR image pulls and
 - Permissions allow CloudWatch logging.



4.3 ECR Validation

Navigate to **Amazon ECR** console and select **Private repository**. Verify the following:

- ECR repository exists and matches terraform variable.
- No image is pushed in to the repository yet.

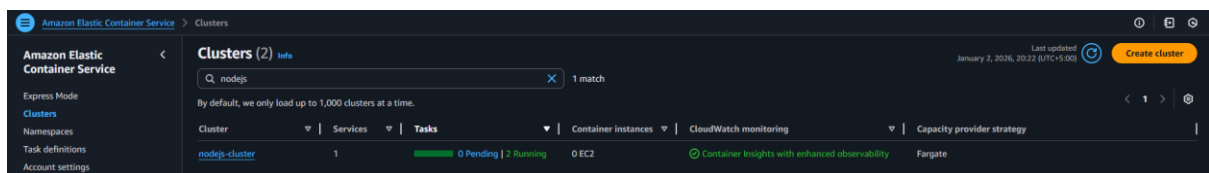


4.4 ECS Validation

1. ECS Cluster

Navigate to **ECS** console and select **Clusters**. Verify the following:

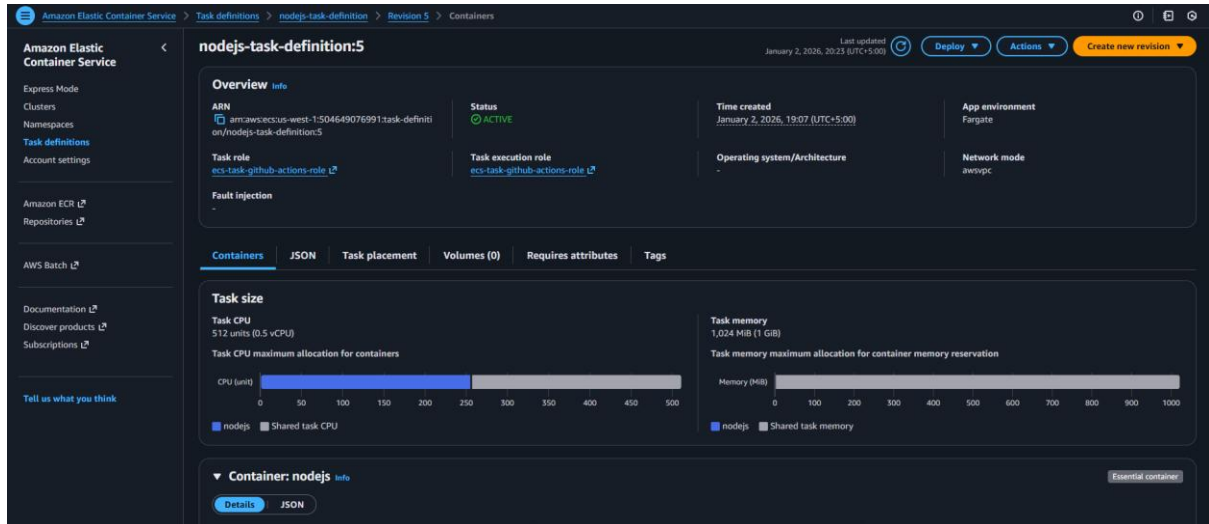
- ECS cluster exists.
- Container Insights enabled.



2. Task Definition

Navigate to **Task Definitions** section. Verify the following:

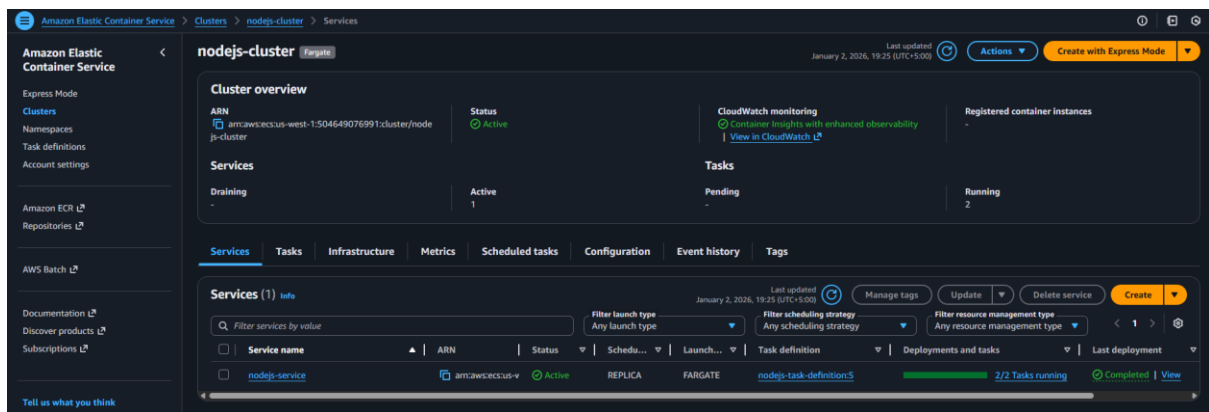
- Task definition exists.
- Uses Fargate launch type.
- References correct ECR image.
- CPU, memory, and port mappings are correct.



3. ECS Service

Navigate to **Services** inside the ECS cluster. Verify the following:

- Desired count is set to 2 as defined in terraform code.
- Tasks are in a **stopped** state (there is no docker image).
- Tasks are placed in private subnets.
- Service is associated with the ALB target group.



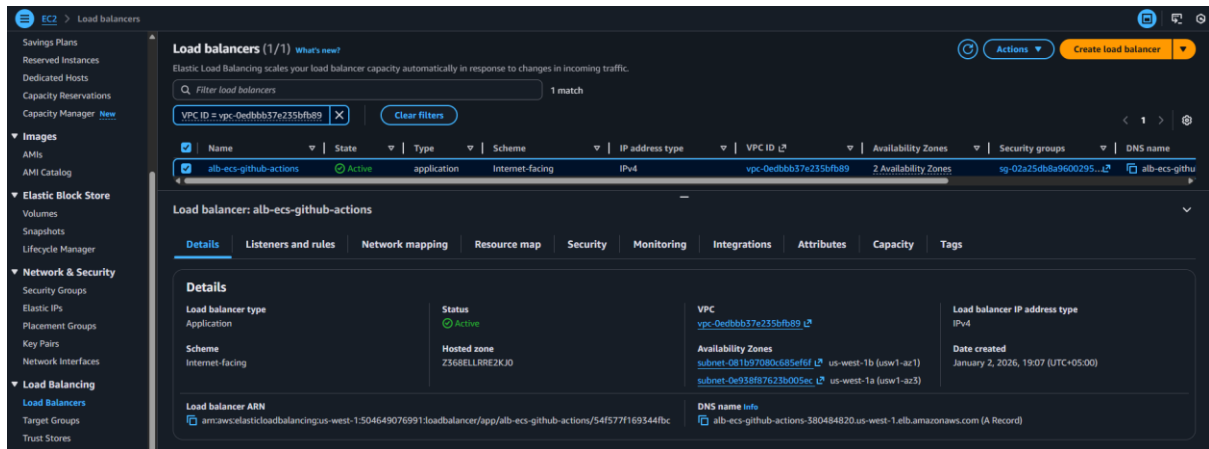
4.6 Application Load Balancer Validation

1. Load Balancer

Navigate to **EC2 console**. Select **Load Balancers** located on the left navigation bar and verify the following:

- Application Load Balancer exists with the name: **alb-ecs-github-actions**

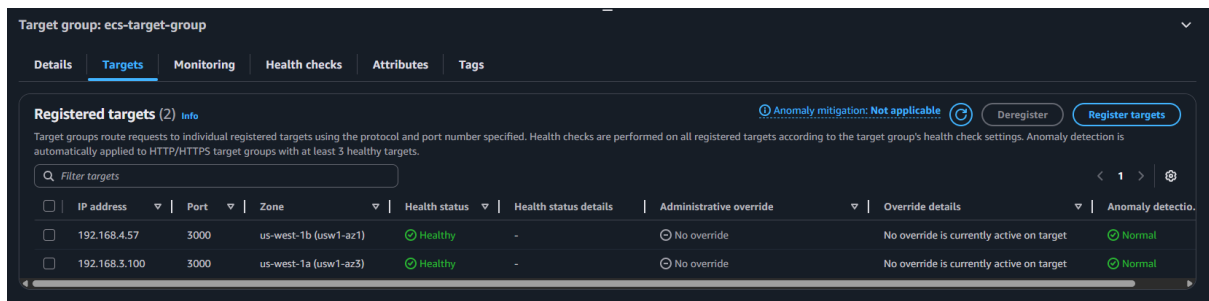
- Assigned to public subnets.
- **Listener:** HTTP:80 → Target Group
- **Security group:** ALB-SG.



2. Target Group

Within the EC2 console, select **Target Groups** located on the left navigation bar. Verify the following:

- **Target type:** IP
- **Port:** 3000.
- Targets are registered after running the pipeline.



5. GitHub Actions CI/CD Pipeline

This section describes the implementation of a cloud-native CI/CD pipeline using GitHub Actions with AWS ECS and Amazon ECR. The pipeline replaces a traditional Jenkins-based setup and follows AWS security best practices by using **OIDC-based IAM role** assumption, eliminating long-lived AWS credentials.

5.1 GitHub Actions Workflow Overview

The CI/CD pipeline is executed on **GitHub-hosted runners**. The workflow is responsible for the following:

- Authenticating to AWS using OIDC
- Building a Docker image for the Node.js application
- Pushing the image to Amazon ECR
- Deploying the updated image to an ECS service

Key Characteristics

- No self-hosted runners
- No stored AWS credentials
- Short-lived authentication using OIDC
- Manual execution for controlled deployments

Workflow Location

GitHub workflows YAML file(s) must be in the `.github/workflows/<YAML file>` within the project root directory. Example: **`.github/workflows/deploy.yml`**

This file can have any name, but it must be in YAML format.

```

! deploy.yml X
.github > workflows > ! deploy.yml
1  name: Build, Test, and Deploy to AWS ECS
2
3  on:
4    workflow_dispatch:
5
6  permissions:
7    id-token: write
8    contents: read
9
10 env:
11   AWS_REGION: ${vars.AWS_REGION}
12   ECR_REPO: ${vars.ECR_REPO}
13   OIDC_ROLE: ${vars.OIDC_ROLE}
14   IMAGE_TAG: latest
15   ECS_CLUSTER: nodejs-cluster
16   ECS_SERVICE: nodejs-service
17
18 jobs:
19   deploy:
20     runs-on: ubuntu-latest
21
22     steps:
23       # Checkout source code
24       - name: Checkout code
25         uses: actions/checkout@v4
26
27       # Configure AWS credentials via OIDC
28       - name: Configure AWS credentials
29         uses: aws-actions/configure-aws-credentials@v4
30         with:
31           role-to-assume: ${env.OIDC_ROLE}
32           aws-region: ${env.AWS_REGION}
33
34       # Login to Amazon ECR
35       - name: Login to Amazon ECR
36         uses: aws-actions/amazon-ecr-login@v2
37
38       # Build Docker image
39       - name: Build Docker image
40         run: |
41           docker build -t nodejs-app:${IMAGE_TAG} .
42
43       # Tag and push image to ECR
44       - name: Push Docker image to ECR
45         run: |
46           docker tag nodejs-app:${IMAGE_TAG} $ECR_REPO:${IMAGE_TAG}
47           docker push $ECR_REPO:${IMAGE_TAG}
48
49       # Deploy to ECS
50       - name: Deploy to ECS
51         run: |
52           aws ecs update-service \
53             --cluster $ECS_CLUSTER \
54             --service $ECS_SERVICE \
55             --force-new-deployment \
56             --region $AWS_REGION
57

```

The following repository variables were added as well.

Name	Value	Last updated
 AWS_REGION	us-west-1	9 minutes ago  
 ECR_REPO	504649076991.dkr.ecr.us-west-1.amazonaws.c...	8 minutes ago  
 OIDC_ROLE	arn:aws:iam::504649076991:role/github-actio...	7 minutes ago  

5.2 OIDC-Based Authentication with AWS

To securely authenticate GitHub Actions with AWS, the pipeline uses **OpenID Connect (OIDC)**. Traditional CI/CD pipelines store AWS access keys as secrets. This project avoids that by using Federated identity, short-lived credentials, and IAM role assumption via STS.

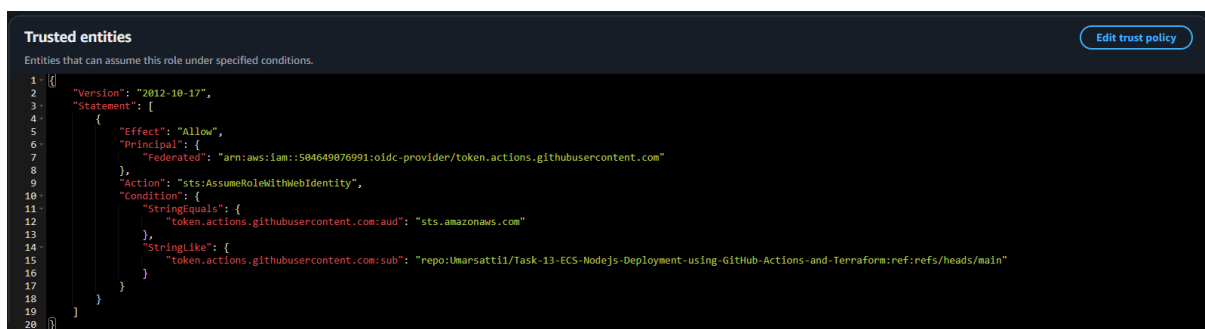
How Authentication Works

- GitHub Actions requests an OIDC token during workflow execution
- AWS verifies the token against the IAM OIDC provider
- AWS STS allows the workflow to assume a dedicated IAM role
- Temporary credentials are issued for the duration of the job

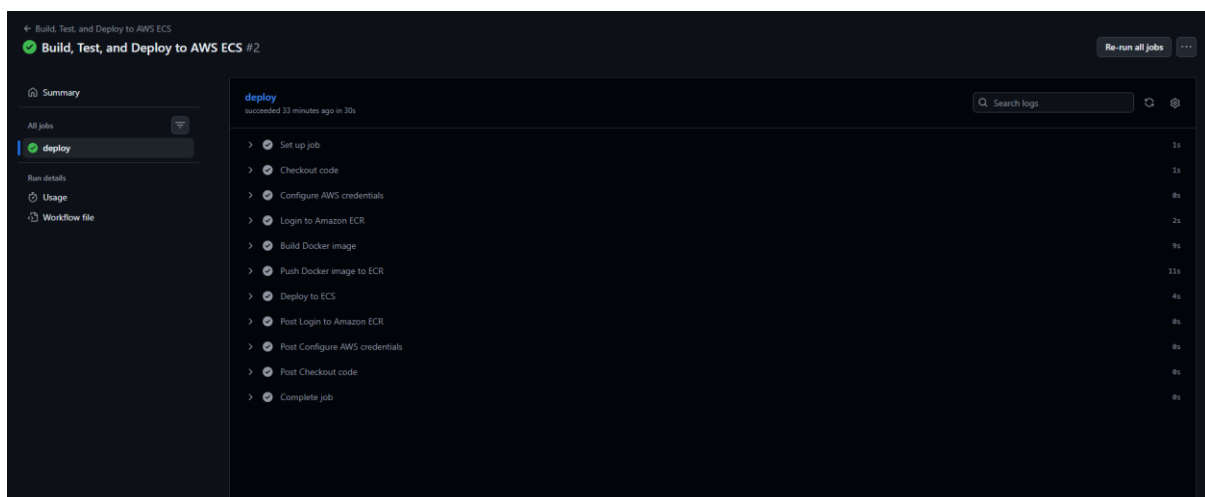
IAM Trust Conditions

The IAM role trust policy ensures:

- Only this GitHub repository can assume the role
- Only the main branch is allowed
- The correct OIDC audience (sts.amazonaws.com) is used
- Enforces **least privilege** and prevents unauthorized access.



5.3 Workflow Execution Steps



The GitHub Actions workflow performs the following steps sequentially:

Set up Job

```
1 Current runner version: '2.300.0'
2 ▶ Runner Image Provisioner
8 ▶ Operating System
12 ▶ Runner Image
17 ▶ GITHUB_TOKEN Permissions
20 Secret source: Actions
21 Prepare workflow directory
22 Prepare all required actions
23 Getting action download info
24 Download action repository 'actions/checkout@v4' (SHA:34e114876b0b11c390a56381ad16ebd13914f8d5)
25 Download action repository 'aws-actions/configure-aws-credentials@v4' (SHA:7474bc4690c29a8392af63c5b98e7449536d5c3a)
26 Download action repository 'aws-actions/amazon-ecr-login@v2' (SHA:062b18b96a7aff07104dc91bc00c4c1a7945b076)
27 Complete job name: deploy
```

Step 1: Checkout Source Code

- Pulls the latest code from the main branch
- Ensures the Docker build uses the most recent application version

```
1 ▶ Run actions/checkout@v4
23 Syncing repository: Umarsatt1/Task-13-ECS-Nodejs-Deployment-using-GitHub-Actions-and-Terraform
24 ▶ Getting Git version info
28 Temporarily overriding HOME='/home/runner/work/_temp/caf0ce17-c371-4a15-856d-58dba6044c8c' before making global git config changes
29 Adding repository directory to the temporary git global config as a safe directory
30 /usr/bin/git config --global --add safe.directory /home/runner/work/Task-13-ECS-Nodejs-Deployment-using-GitHub-Actions-and-Terraform/Task-13-ECS-Nodejs-Deployment-using-GitHub-Actions-and-Terraform
31 Deleting the contents of '/home/runner/work/Task-13-ECS-Nodejs-Deployment-using-GitHub-Actions-and-Terraform/Task-13-ECS-Nodejs-Deployment-using-GitHub-Actions-and-Terraform'
32 ▶ Initializing the repository
49 ▶ Disabling automatic garbage collection
51 ▶ Setting up auth
59 ▶ Fetching the repository
63 ▶ Determining the checkout info
64 /usr/bin/git sparse-checkout disable
65 /usr/bin/git config --local --unset-all extensions.worktreeConfig
66 ▶ Checking out the ref
70 /usr/bin/git log -1 --format=%H
71 acebac40b0225936a2a526f1fdc524072352db
```

Step 2: Configure AWS Credentials (OIDC)

- Uses the aws-actions/configure-aws-credentials action
- Assumes the IAM role via sts:AssumeRoleWithWebIdentity
- No AWS secrets are stored in GitHub

```
1 ▶ Run aws-actions/configure-aws-credentials@v4
14 Assuming role with OIDC
15 Authenticated as assumedRoleId AROAXX73N2D7XNPOHF4ZE:GitHubActions
```

Step 3: Authenticate to Amazon ECR

- Logs in to Amazon ECR using temporary credentials
- Allows Docker image push access

```
1 ▶ Run aws-actions/amazon-ecr-login@v2
17 Logging into registry 504649076991.dkr.ecr.us-west-1.amazonaws.com
```

Step 4: Build Docker Image

- Builds the Node.js application image using the Dockerfile
- Tags the image with latest

```
Build Docker image 9s
1 ▶ Run docker build -t nodejs-app:${IMAGE_TAG} .
15 #0 building with "default" instance using docker driver
16
17 #1 [internal] load build definition from Dockerfile
18 #1 transferring dockerfile: 534B done
19 #1 DONE 0.0s
20
21 #2 [auth] library/node:pull token for registry-1.docker.io
22 #2 DONE 0.0s
23
24 #3 [internal] load metadata for docker.io/library/node:20-alpine
25 #3 DONE 0.7s
26
27 #4 [internal] load .dockerignore
28 #4 transferring context: 2B done
29 #4 DONE 0.0s
30
31 #5 [build 1/4] FROM docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4b095c71e7d704537c9d9272f488ac03a370d448
32 #5 resolve docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4b095c71e7d704537c9d9272f488ac03a370d448
33 #5 ...
34
```

```
98 #12 [stage-1 5/5] COPY public ./public
99 #12 DONE 0.0s
100
101 #13 exporting to image
102 #13 exporting layers
103 #13 exporting layers 0.3s done
104 #13 writing image sha256:2b4fef18034b6095432c565a73524998809e01d3b77938e5e5a60702446978cb done
105 #13 naming to docker.io/library/nodejs-app:latest done
106 #13 DONE 0.3s
```

Step 5: Push Image to ECR

- Pushes the built image to the ECR repository
- The image becomes immediately available for ECS

```
Push Docker image to ECR 11s
1 ▶ Run docker tag nodejs-app:${IMAGE_TAG} $ECR_REPO:${IMAGE_TAG}
16 The push refers to repository [504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-app]
17 0a59adc8b05a: Preparing
18 4fe22976e3de: Preparing
19 2d7442fa202b: Preparing
20 8efd771d54b7: Preparing
21 6bc78693c259: Preparing
22 571b9a150b87: Preparing
23 c7e70c519c8c: Preparing
24 7bb20cf5ef67: Preparing
25 571b9a150b87: Waiting
26 c7e70c519c8c: Waiting
27 7bb20cf5ef67: Waiting
28 8efd771d54b7: Pushed
29 0a59adc8b05a: Pushed
30 4fe22976e3de: Pushed
31 6bc78693c259: Pushed
32 2d7442fa202b: Pushed
33 571b9a150b87: Pushed
34 7bb20cf5ef67: Pushed
35 c7e70c519c8c: Pushed
36 latest: digest: sha256:4410767dcb6d7bb1c8ae8185738c8c4d4f71811629a2951561b0f07a5f04191 size: 1988
```

Step 6: Deploy to ECS

- Triggers a new ECS deployment
- Forces the ECS service to pull the latest image
- Performs a rolling update with zero downtime

```
Deploy to ECS 4s
1 ▶ Run aws ecs update-service \
19 {
20   "service": {
21     "serviceName": "nodejs-service",
22     "clusterArn": "arn:aws:ecs:us-west-1:504649076991:cluster/nodejs-cluster",
23     "loadBalancers": [
24       {
25         "targetGroupArn": "arn:aws:elasticloadbalancing:us-west-1:504649076991:targetgroup/ecs-target-group/705a63198aabbdf0",
26         "containerName": "nodejs",
27         "containerPort": 3000
28       }
29     ]
30   }
}
```

Post Deploy

```
✓ Post Login to Amazon ECR 0s
1 Post job cleanup.
2 Logging out of registry 504649076991.dkr.ecr.us-west-1.amazonaws.com

✓ Post Configure AWS credentials 0s
1 Post job cleanup.

✓ Post Checkout code 0s
1 Post job cleanup.
2 /usr/bin/git version
3 git version 2.52.0
4 Temporarily overriding HOME="/home/runner/work/_temp/Sec593cb-e5ab-4ffc-8f52-896228617e5e" before making global git config changes
5 Adding repository directory to the temporary git global config as a safe directory
6 /usr/bin/git config --global --add safe.directory /home/runner/work/Task-13-ECS-Nodejs-Deployment-using-Github-Actions-and-Terraform/Task-13-ECS-Nodejs-Deployment-using-Github-Actions-and-Terraform
7 /usr/bin/git config --local --name-only --get-regexp core.sshCommand
8 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'core.sshCommand' && git config --local --unset-all 'core.sshCommand' || :"
9 /usr/bin/git config --local --name-only --get-regexp http.https://github.com/.extraheader
10 http.https://github.com/.extraheader
11 /usr/bin/git config --local --unset-all http.https://github.com/.extraheader
12 /usr/bin/git submodule foreach --recursive sh -c "git config --local --name-only --get-regexp 'http.https://github.com/.extraheader' && git config --local --unset-all 'http.https://github.com/.extraheader' || :"
13 /usr/bin/git config --local --name-only --get-regexp includeIf\.\gitdir:
14 /usr/bin/git submodule foreach --recursive git config --local --show-origin --name-only --get-regexp remote.origin.url

✓ Complete job 0s
1 Cleaning up orphan processes
```

5.4 Manual Workflow Trigger

For better control and safety, the pipeline is configured to run **manually** instead of automatically on every push. This is configured using `workflow_dispatch` function.

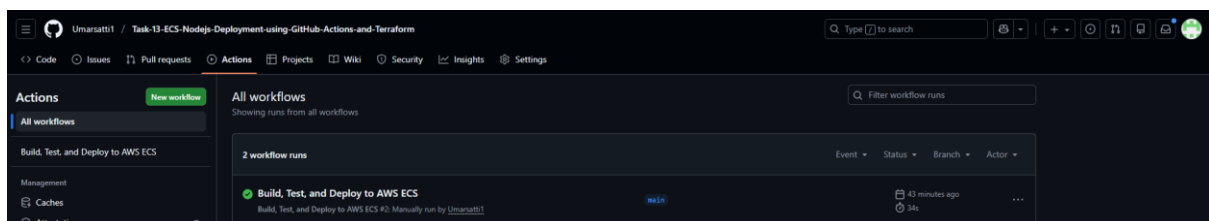
```
on:
  workflow_dispatch:
```

Benefits of Manual Execution

- Prevents accidental deployments
- Suitable for production environments
- Allows controlled releases and demonstrations

How to Run the Pipeline Manually

- Navigate to the GitHub repository and click **Actions** tab.
- Select the deployment workflow and click **Run workflow**.
- Choose the **main** branch, then click **Run workflow**,

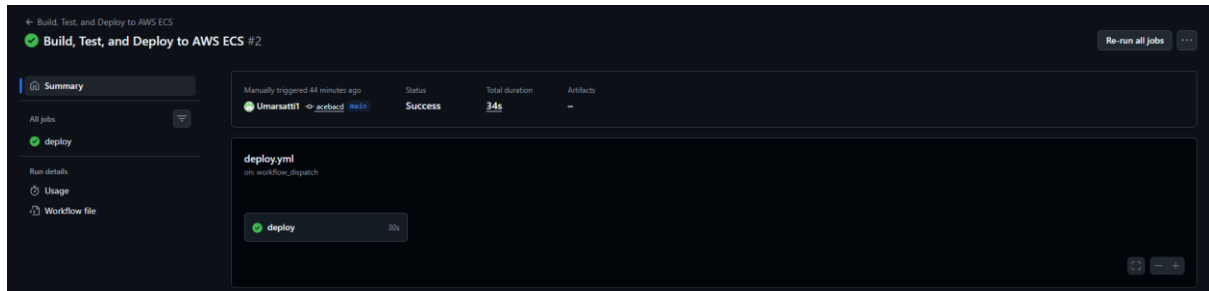


5.5 Validation

After the workflow completes successfully, the deployment is validated using multiple methods.

1. GitHub Actions Validation

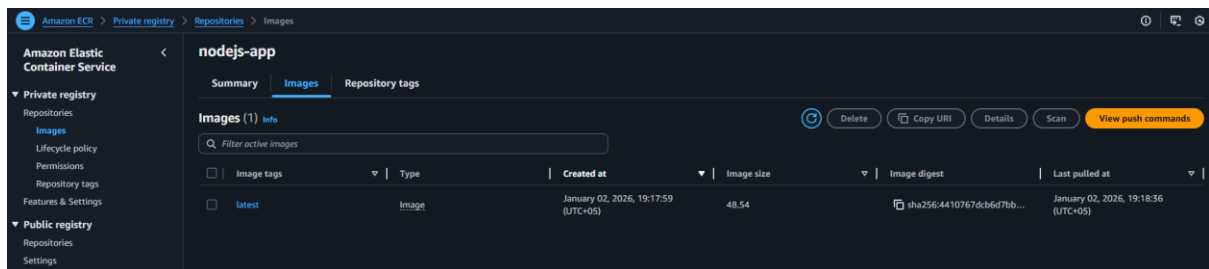
- Workflow status shows **Success**
- All steps completed without errors



2. Amazon ECR Validation

Navigate to ECR console. Under Private registry, click Repositories. Click the repository named **nodejs-app**. An image is displayed with the **latest** image tag.

- New Docker image visible in the ECR repository
- Image tag updated (latest)



3. Amazon ECS Validation

Navigate to ECR console. Under Private registry, click Repositories. Click the repository named **nodejs-app**. An image is displayed with the **latest** image tag.

- ECS service shows a new deployment
- Tasks are running successfully
- Desired count is maintained
- Events showcasing steady state and health of ECS tasks

The images below display ECS service Health, Tasks, Deployment, and Events tab.

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

nodejs-service

January 2, 2026, 20:24 (UTC+5:00)

Delete service

Update service

Service overview

Status: Active

Tasks (2 Desired): 0 Pending | 2 Running

Task definition: revision: nodejs-task-definition:5

Deployment status: Success

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Event history

Tags

Status

Service name: nodejs-service

Service ARN: arn:aws:ecs:us-west-1:504649076991:service/nodejs-cluster/nodejs-service

Deployments current state: 2 Completed tasks

Created at: January 2, 2026, 19:11 (UTC+5:00)

Health check grace period: 0 seconds

Load balancer target health

Container name:port

Target group: ecs-target-group

Target health: 2 Healthy | 0 Unhealthy

Load balancer: alb-ecs-github-actions

Load balancer type: Application Load Balancer

Amazon Elastic Container Service

Express Mode

Clusters

Namespaces

Task definitions

Account settings

Amazon ECR

Repositories

AWS Batch

Documentation

Discover products

Subscriptions

Tell us what you think

nodejs-service

January 2, 2026, 19:24 (UTC+5:00)

Delete service

Update service

Service overview

Status: Active

Tasks (2 Desired): 0 Pending | 2 Running

Task definition: revision: nodejs-task-definition:5

Deployment status: Success

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Event history

Tags

Tasks (1/2)

Filter tasks by property or value

Filter desired status: Running

Filter launch type: Any launch type

Task

Last status

Desired status

Task definition

Health status

Created at

Started by

Started at

Container

234609d12aef4b289390ae7d4e972454

Running

Running

nodejs-task-definition:5

Unknown

4 minutes ago

ecs-svc/8255803904623...

4 minutes ago

-

954151cf9008430b9d5311168cf412f3

Running

Running

nodejs-task-definition:5

Unknown

6 minutes ago

ecs-svc/8255803904623...

6 minutes ago

-

Health and metrics

Tasks

Logs

Deployments

Events

Configuration and networking

Service auto scaling

Event history

Tags

Last deployment

Deployment ID: 0ivB6XrwWu0sFHcbvYl8a

Deployment status: Success

Deployment controller type: ECS

Deployment strategy: Rolling update

Min and max running tasks: 50% min and 100% max

Deployment duration: 3 minutes, 33 seconds

Created at: January 2, 2026, 19:18 (UTC+5:00)

Started at: January 2, 2026, 19:18 (UTC+5:00)

Stopped at: -

Finished at: January 2, 2026, 19:21 (UTC+5:00)

Service revisions (2)

A service revision includes the number of tasks involved in the service deployment. You can choose to view details for all service revisions created on or after October 24, 2024.

Service revision	Revision type	Task count at deployment completion	Task definition: revision
8255803904623028835 View tasks	Target	2 Requested 2 Running 0 Pending	nodejs-task-definition:5
9383390855903474633 View tasks	Source	0 Requested 0 Running 0 Pending	nodejs-task-definition:5

Events (16)

January 2, 2026, 19:23 (UTC+5:00)

Event type: All events

Filter events by value

Started at

Message

January 2, 2026, 19:21 (UTC+5:00): service nodejs-service has reached a steady state.

January 2, 2026, 19:21 (UTC+5:00): service nodejs-service deployment ecs-svc/8255803904623028835 deployment completed.

January 2, 2026, 19:21 (UTC+5:00): service nodejs-service registered 1 targets in target-group ecs-target-group

January 2, 2026, 19:20 (UTC+5:00): service nodejs-service has started 1 tasks: task 234609d12aef4b289390ae7d4e972454.

January 2, 2026, 19:19 (UTC+5:00): (service nodejs-service, taskSet ecs-svc/9383390855903474633) has begun draining connections on 1 tasks.

January 2, 2026, 19:19 (UTC+5:00): service nodejs-service deregistered 1 targets in target-group ecs-target-group

January 2, 2026, 19:19 (UTC+5:00): service nodejs-service has stopped 1 running tasks: task 5046231c5d8041a5a7a8cf7568277962.

January 2, 2026, 19:18 (UTC+5:00): service nodejs-service registered 1 targets in target-group ecs-target-group

January 2, 2026, 19:18 (UTC+5:00): service nodejs-service has started 1 tasks: task 954151cf9008430b9d5311168cf412f3.

January 2, 2026, 19:18 (UTC+5:00): service nodejs-service has started 1 tasks: task 5046231c5d8041a5a7a8cf7568277962.

January 2, 2026, 19:17 (UTC+5:00): service nodejs-service was unable to place a task. Reason: CannotPullContainerError: pull image manifest has been retried 7 time(s): failed to resolve ref 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: not found.

January 2, 2026, 19:16 (UTC+5:00): service nodejs-service has started 1 tasks: task 52b15477cefa4ad2ab38fe8808b68f4f.

January 2, 2026, 19:15 (UTC+5:00): service nodejs-service was unable to place a task. Reason: CannotPullContainerError: pull image manifest has been retried 7 time(s): failed to resolve ref 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: not found.

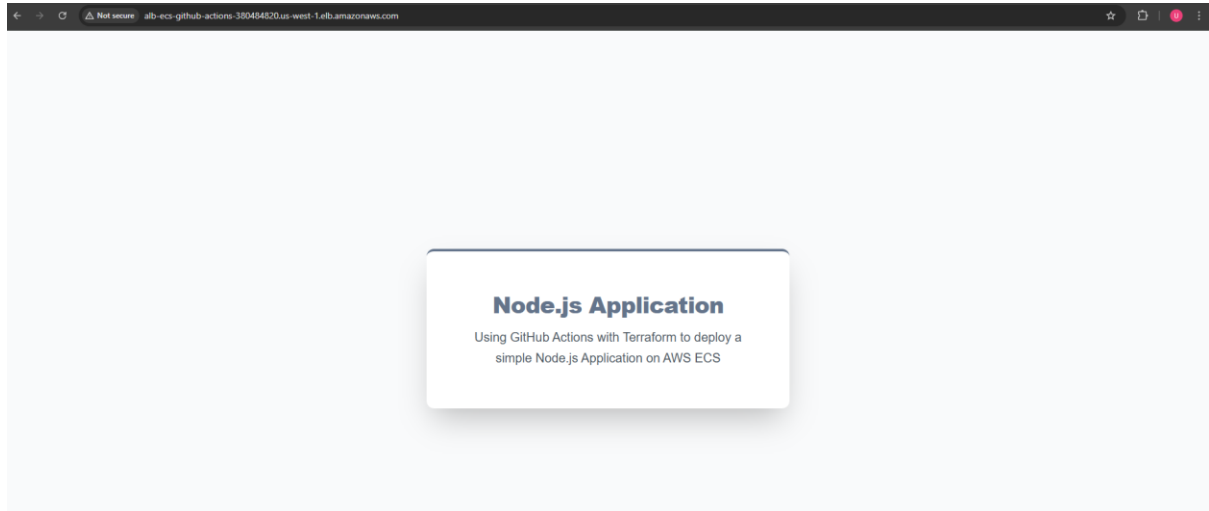
January 2, 2026, 19:13 (UTC+5:00): service nodejs-service has started 1 tasks: task 3560494fb9f64bd18aba080d13211f8e.

January 2, 2026, 19:13 (UTC+5:00): service nodejs-service was unable to place a task. Reason: CannotPullContainerError: pull image manifest has been retried 7 time(s): failed to resolve ref 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: 504649076991.dkr.ecr.us-west-1.amazonaws.com/nodejs-applatest: not found.

January 2, 2026, 19:11 (UTC+5:00): service nodejs-service has started 1 tasks: task 73fd8d129d3b4a6586d1a8a3f03abff0.

4. Application Validation

- Application accessible via Application Load Balancer DNS
 - ***alb-ecs-github-actions-380484820.us-west-1.elb.amazonaws.com***
- Confirms end-to-end deployment success

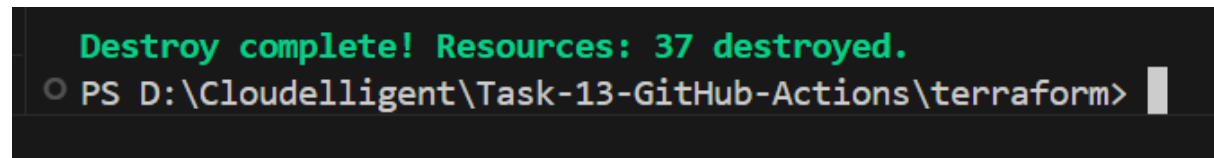


Summary

This GitHub Actions CI/CD pipeline demonstrates a modern, secure approach to application deployment on AWS. By combining GitHub-hosted runners, OIDC-based authentication, and ECS rolling deployments, the pipeline eliminates credential management overhead while maintaining strong security and operational efficiency.

6. Clean Up

The **`terraform destroy --auto-approve`** command was executed to remove all AWS resources created by Terraform. This ensures that no infrastructure is left running, helping prevent unnecessary costs. The successful completion message confirms that all **37** resources were **destroyed**, as shown in the screenshot.

A screenshot of a terminal window with a dark background. The first line shows the message "Destroy complete! Resources: 37 destroyed." in green text. The second line shows a PowerShell prompt "PS D:\Cloudelligent\Task-13-GitHub-Actions\terraform>" in white text, followed by a white cursor bar.

```
Destroy complete! Resources: 37 destroyed.  
PS D:\Cloudelligent\Task-13-GitHub-Actions\terraform>
```

7. Troubleshooting

Problem Description

The GitHub Actions workflow failed during **Step 2: Configure AWS Credentials** with the following error:

```
Error: Could not assume role with OIDC: Not authorized to perform  
sts:AssumeRoleWithWebIdentity
```

Due to this failure, the pipeline stopped before the Docker build and ECS deployment stages.

Root Cause

The IAM role trust policy used for GitHub Actions referenced an incorrect **GitHub repository** name in the `token.actions.githubusercontent.com:sub` condition. The repository name belonged to an older Jenkins-based project, causing AWS STS to deny the OIDC role assumption request.

Solution

The issue was resolved by updating the IAM trust policy to reference the correct **GitHub repository** name and reapplying the Terraform configuration. After rerunning the workflow, GitHub Actions successfully assumed the IAM role and completed the ECS deployment.